부록 · 파이썬으로 DB 다루기

PyMySQL — 넣기 · 읽기 · 고치기 · 지우기

2-5 참고의 「기본 5가지」 다음 단계입니다. 연습용 표 py_memo 를 만들어 네 가지 동작을 한 번씩 해 봅니다. 수업용 표(products 등)는 건드리지 않습니다.

준비

```bash
# 2-1 의 de-mysql(또는 2-3 의 de-mongo)이 켜져 있어야 합니다
docker start de-mysql

# 가상환경을 켜고 필요한 것만 설치합니다
cd ~/data-eng/week2 && source .venv/bin/activate
pip install pymysql sqlalchemy pandas python-dotenv pymongo
python3 -c "import sys; print(sys.prefix)"     # .venv 로 끝나야 가상환경에 설치된 것
```

코드

```python
# py_mysql_crud.py — PyMySQL 로 넣기·읽기·고치기·지우기 (2-1 의 de-mysql · shop DB)
import pymysql

conn = pymysql.connect(host='127.0.0.1', port=3306, user='root', password='dataeng123',
                       database='shop', charset='utf8mb4',
                       cursorclass=pymysql.cursors.DictCursor)

with conn:                                      # 블록이 끝나면 연결을 닫는다
    with conn.cursor() as cur:
        # 연습용 표 — 몇 번 돌려도 같은 결과가 나오게 지우고 새로 만든다
        cur.execute("DROP TABLE IF EXISTS py_memo")
        cur.execute("""
            CREATE TABLE py_memo (
              id    INT AUTO_INCREMENT PRIMARY KEY,
              title VARCHAR(100) NOT NULL,
              done  TINYINT NOT NULL DEFAULT 0
            )""")

        # 넣기 — lastrowid 로 방금 생긴 id 를 받는다
        cur.execute("INSERT INTO py_memo (title) VALUES (%s)", ('MySQL 복습',))
        print('새 id:', cur.lastrowid)
        cur.executemany("INSERT INTO py_memo (title) VALUES (%s)", [('pandas 읽기',), ('MongoDB
적재',)])
        conn.commit()

        # 읽기
        cur.execute("SELECT id, title, done FROM py_memo ORDER BY id")
        for row in cur.fetchall():
            print(row)

        # 고치기 — execute 가 돌려주는 수가 바뀐 행 수
        n = cur.execute("UPDATE py_memo SET done = 1 WHERE title = %s", ('MySQL 복습',))
        conn.commit()
        print('고친 행:', n)

        # 지우기
        n = cur.execute("DELETE FROM py_memo WHERE done = %s", (1,))
        conn.commit()
        print('지운 행:', n)

        cur.execute("SELECT COUNT(*) AS n FROM py_memo")
        print('남은 행:', cur.fetchone()['n'])
```

```bash
python3 py_mysql_crud.py
```

실행 결과 (실측)

```text
새 id: 1
{'id': 1, 'title': 'MySQL 복습', 'done': 0}
{'id': 2, 'title': 'pandas 읽기', 'done': 0}
{'id': 3, 'title': 'MongoDB 적재', 'done': 0}
고친 행: 1
지운 행: 1
남은 행: 2
```

짚을 점

- cursorclass=DictCursor — 결과가 딕셔너리라 row['title'] 처럼 칸 이름으로 꺼냅니다. 빼면 튜플이라 row[1] 처럼 순서로 꺼내야 합니다.
- cur.lastrowid — 방금 넣은 행의 AUTO_INCREMENT 번호입니다. 주문을 넣고 그 번호로 주문 상세를 넣을 때 씁니다.
- cur.execute(...) 가 돌려주는 수 — 영향을 받은 행 수입니다. UPDATE·DELETE 뒤에 0 이면 조건에 맞는 행이 없었던 것입니다.
- with conn: — 블록이 끝나면 연결을 닫습니다. 닫는 것을 잊어 연결이 쌓이는 일을 막습니다.

commit 을 빠뜨리면

INSERT 뒤에 commit 없이 연결을 닫으면 넣은 행이 사라집니다. 같은 연결 안에서는 보이기 때문에 알아채기 어렵습니다 (실측: 같은 연결에서 3건 → 다시 연결하면 2건).

그래서 「넣었는데 다른 창에서 안 보인다」면 commit 부터 의심합니다. 바꾸는 명령(INSERT·UPDATE·DELETE) 뒤에는 conn.commit() 을 붙이는 습관을 들입니다.
